

Large-Scale and Multi-Structured Databases

Tonight's Call

Francesco Della Maggiora

Alessandro Xavier Battisti

Martina Iacoponi

Application Highlights

The aim of this application is to offer users a quick rundown of what events take place in their area (and beyond) and allow event organizers to publish their events

- *Users have access to a map to visualize every event based on their location*
- *Users have access to a browser to search for events based on topic, date or location*
- *Users can state their presence at events*
- *Users can review events and assign a numerical score to them*
- *Users can visualize events' details and ratings*
- *Organizers can create new events and publish them*
- *Organizers can work together in organizations and arrange events as a group*
- *Organizers or Organizing Groups can see the statistics relative to events they have created and decide to publish the results.*

Application non-functional requirements

Performance requirements

- The application must be optimized for speed and usage of data, by processing complex queries with high efficiency.
- The application must support concurrent activity by different users without excessive performance degradation.

Reliability requirements

- The system must prevent permanent data loss.
- The system must guarantee data consistency and fault tolerance.
- Error messages must be clear and descriptive.
- Data consistency is a priority over system availability.

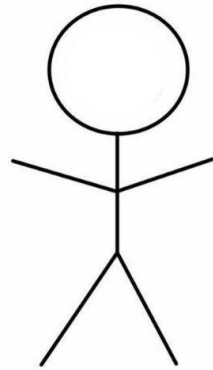
Usability requirements

- The application must be user-friendly.
- The system must allow easy creations of any type of new entity.

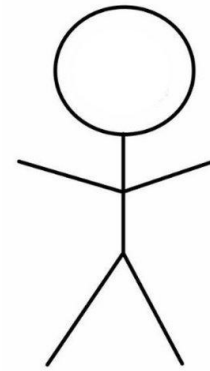
Security requirements

- System features that rely on the usage of user's data must guarantee reliable access through authentication.
- The users' passwords must be encrypted.

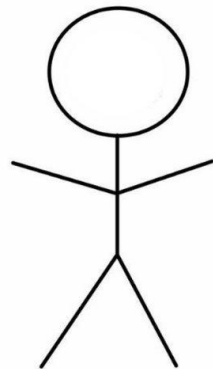
Actors and main mock-ups (i)



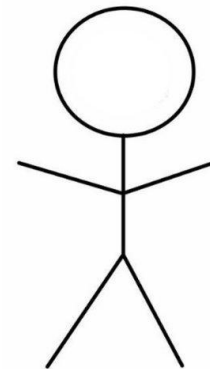
Unregistered User



Registered User



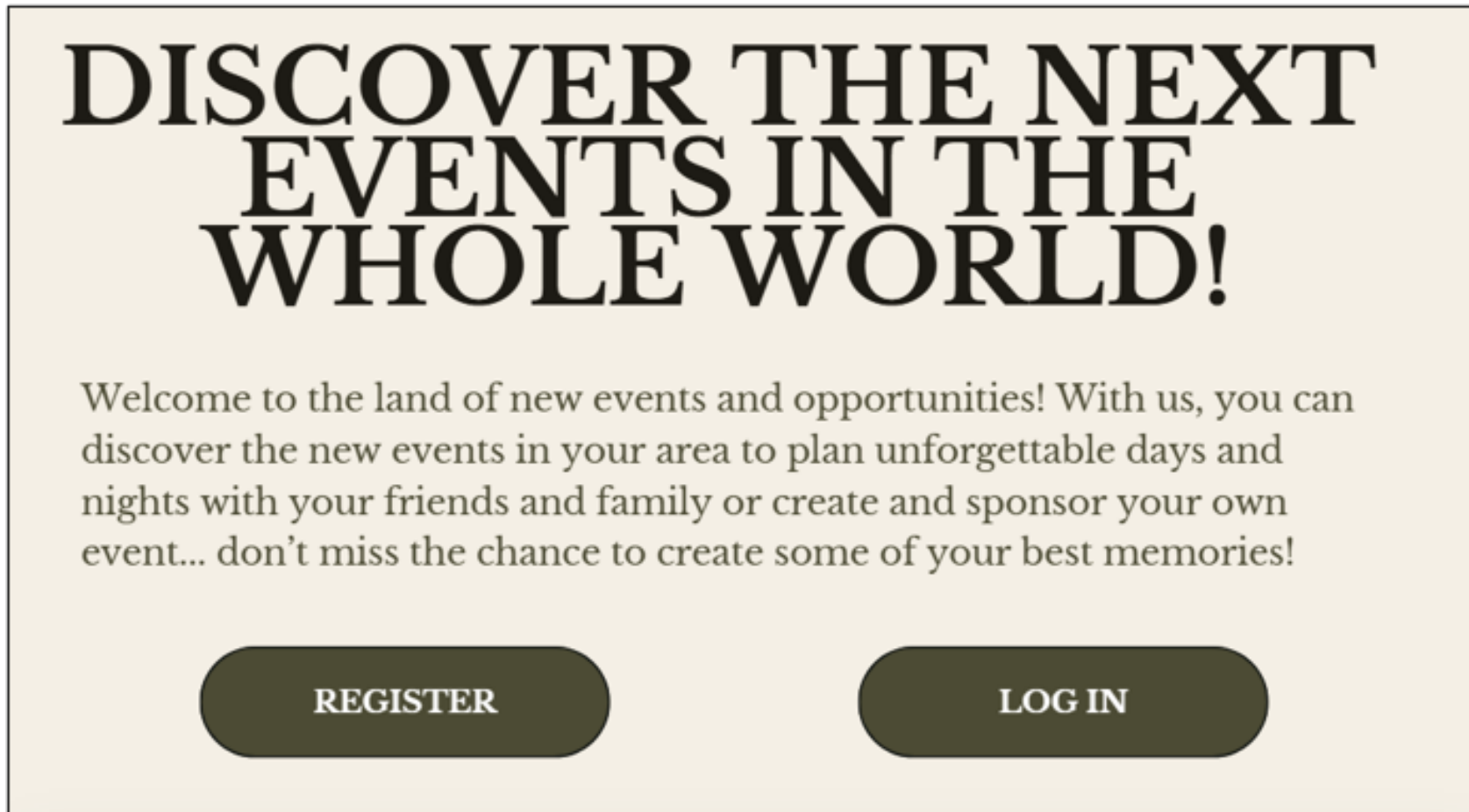
Organizer



Organizing Group

Actors and main mock-ups (ii)

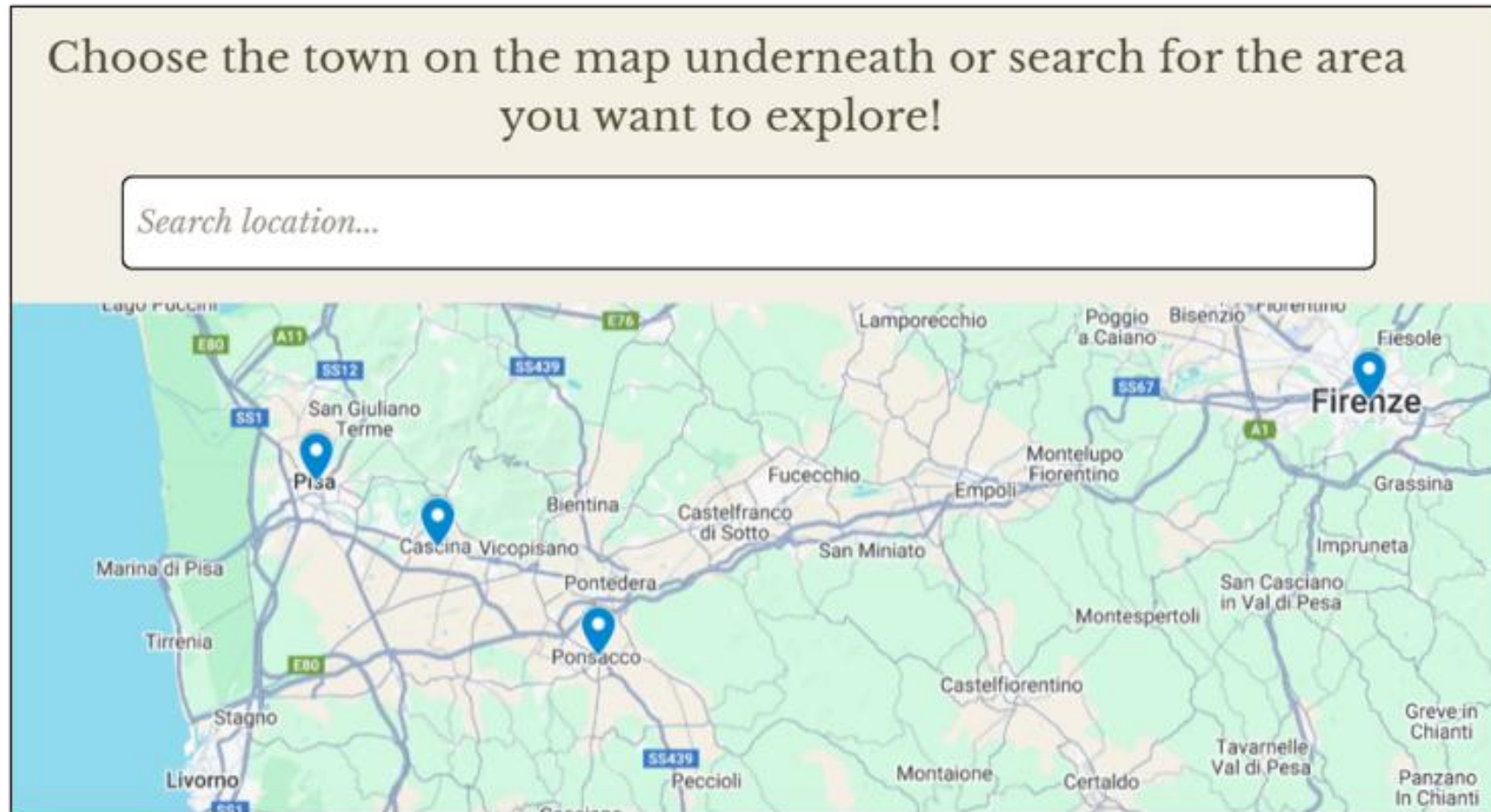
Unregistered User



Main page where the user can create an account or log into their already existing one.

Actors and main mock-ups (iii)

Unregistered user



The user can explore the events by searching for a specific location in the search bar or by selecting a specific event on the map.

Actors and main mock-ups (iv)

Unregistered user

Select a theme or a specific date and explore the next related events you could attend or select a city and access its own events



• Ancona	• Foggia	• Napoli	• Salerno
• Bari	• Forlì	• Novara	• Sassari
• Bergamo	• Genova	• Padova	• Siracusa
• Bologna	• Giugliano in Campania	• Palermo	• Taranto
• Bolzano	• Latina	• Parma	• Terni
• Brescia	• Livorno	• Perugia	• Torino
• Cagliari	• Messina	• Pescara	• Trento
• Catania	• Milano	• Piacenza	• Trieste
• Ferrara	• Modena	• Prato	• Venezia
• Firenze		• Ravenna	• Verona

Page 1 of 3

The user can also explore the list of events in a specific city by clicking on it in the available list or search for events related to a specified theme.

Actors and main mock-ups (v)

Unregistered user

Select a theme or a specific date and explore the next related events you could attend or select a city and access its own events

Search for a theme (try with “music”, “food”, ...)



- Ancona
- Bari
- Bergamo
- Bologna
- Bolzano
- Brescia
- Cagliari
- Catania
- Ferrara
- Firenze
- Foggia
- Forlì
- Genova
- Giugliano in Campania
- Latina
- Livorno
- Messina
- Milano
- Modena

Feb 2026						
Mon	Tue	Wed	Thu	Fri	Sat	Sun
						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	1

The user can also look for events available on a specific date.

Actors and main mock-ups (vi)

Unregistered User / Registered User

FESTIVAL DELLA FRAGOLA

Location

Terricciola (PI), Tuscany, Italy

Price

10-15 € per person

Date

3/5 - 11/5

[JOIN EVENT](#)



Event's page where the user can see its information or state their future presence (unregistered users who try to click the button get redirected to the registration page).

Actors and main mock-ups (vii)

Unregistered User / Registered User

The popular *Festival della fragola* in Terricciola, a little town near Pisa (Italy) is one of the most famous food festivals in Tuscany: you will have the chance to taste amazing farm-to-table strawberries, directly grown and collected on the fields surrounding the area. About the mean of transport, we recommend you travel by car or scooter, so that you will also be able to enjoy the view of the notorious Tuscanian hills where Terricciola was built.

If you need more information about this event, you can read some of the reviews left by other people who attended the festival by clicking the link underneath:

@emcarter_

Loved this festival! Never had tasted such amazing strawberries before! 5 ★

@jw_official

Definetely recommend! Delicious straweberries and friendly people! 4 ★

@parkersmiles

Expected better: price too high and place too crowded... 3 ★

[...more](#)

The user can also view a brief description of the event and reviews left by people who have already attended the event.

Actors and main mock-ups (viii)

Unregistered User



The user can create an account as a participant (namely, a "registered user") or as an organizer of events.

Actors and main mock-ups (ix)

Registered User

John Smith, 45
Cardiff, Wales (UK)
Food Music Nature

UPDATE PROFILE

Explore the next events based on your interest and friendships!

EXPLORE EVENTS

Reviews

<i>Sagra della ciliegia, Lari (Italy)</i>	2 ★
<i>Lucca Summer Festival 2025, Lucca (Italy)</i>	3 ★
<i>Festival della fragola, Terricciola (Italy)</i>	5 ★

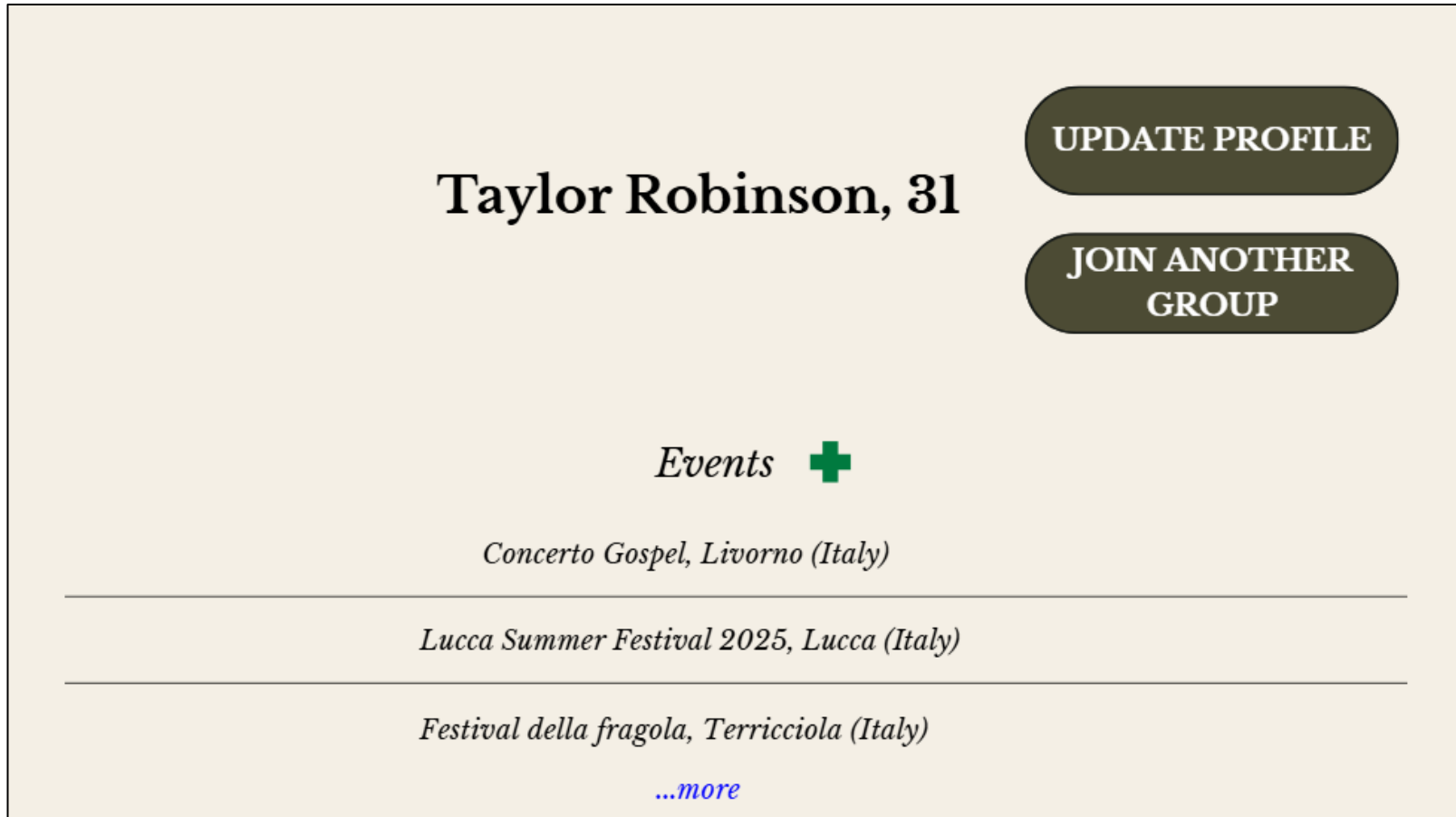
Friend list

<i>@emcarter_</i>	UNFRIEND
<i>@jw_official</i>	UNFRIEND
<i>@parkersmiles</i>	UNFRIEND

User profile: they can update it or explore the suggested events based on their interests and friendships and see their own reviews.

Actors and main mock-ups (x)

Organizer



The mock-up shows a profile for Taylor Robinson, 31. It includes two buttons: 'UPDATE PROFILE' and 'JOIN ANOTHER GROUP'. Below the profile, there is a section for 'Events' with a green plus icon, listing three events: 'Concerto Gospel, Livorno (Italy)', 'Lucca Summer Festival 2025, Lucca (Italy)', and 'Festival della fragola, Terricciola (Italy)'. A link for '...more' is also present.

Taylor Robinson, 31

[UPDATE PROFILE](#)

[JOIN ANOTHER GROUP](#)

Events 

Concerto Gospel, Livorno (Italy)

Lucca Summer Festival 2025, Lucca (Italy)

Festival della fragola, Terricciola (Italy)

[...more](#)

Organizer profile: they can update it, join an Organizing Group, see past events they have created or create new ones.

Actors and main mock-ups (xi)

Organizing Group



3 New Requests
to Join!

Music_queens

UPDATE PROFILE

Events 

Concerto Gospel, Livorno (Italy)

Lucca Summer Festival 2025, Lucca (Italy)

Festival della fragola, Terricciola (Italy)

[...more](#)

Organizers

Mario Rossi @super_mario

Luigi Battisti @best_creator

Camilla Tordello @magic_fun

[...more](#)

Organizing Group profile: they can update it, see who is part of the group, see past events they created or create new ones. They can also see the received join requests.

Actors and main mock-ups (xii)

Organizer / Organizing Group

FESTIVAL DELLA FRAGOLA

Location

Terricciola (PI), Tuscany, Italy

Price

10-15 € per person

Date

3/5 - 11/5



36^a FESTIVAL DELLA FRAGOLA
TERRICCIOLA
5-6 / 12-13 MAGGIO 2018
WWW.FESTIVALDELLAFRAGOLA.IT

UPDATE DATA

Event's page for an Organizer or Organizing Group: the event's data can be updated.

Actors and main mock-ups (xiii)

Organizer / Organizing Group

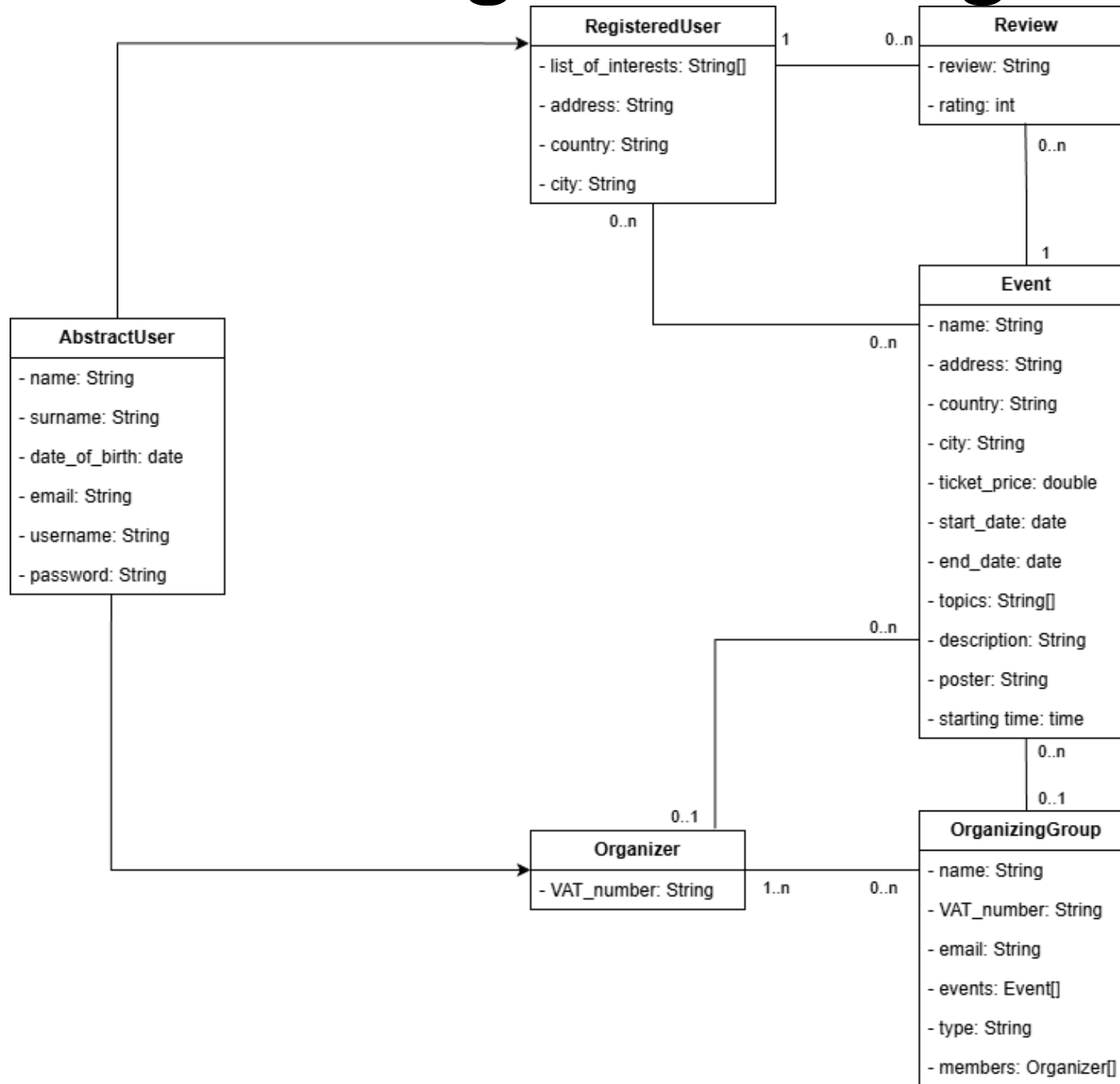
Statistics

- Average age: 53 years old
- Expected proceeds: 15 thousand (€)
- Total number of people: 5000
- People's geographical origin: Pisa: 100, Firenze: 30, Milano: 5
- Average event's rating: 4.3 ★

PUBLISH STATISTICS

Event's statistics: the Organizer or Organizing Group can decide to publish them.

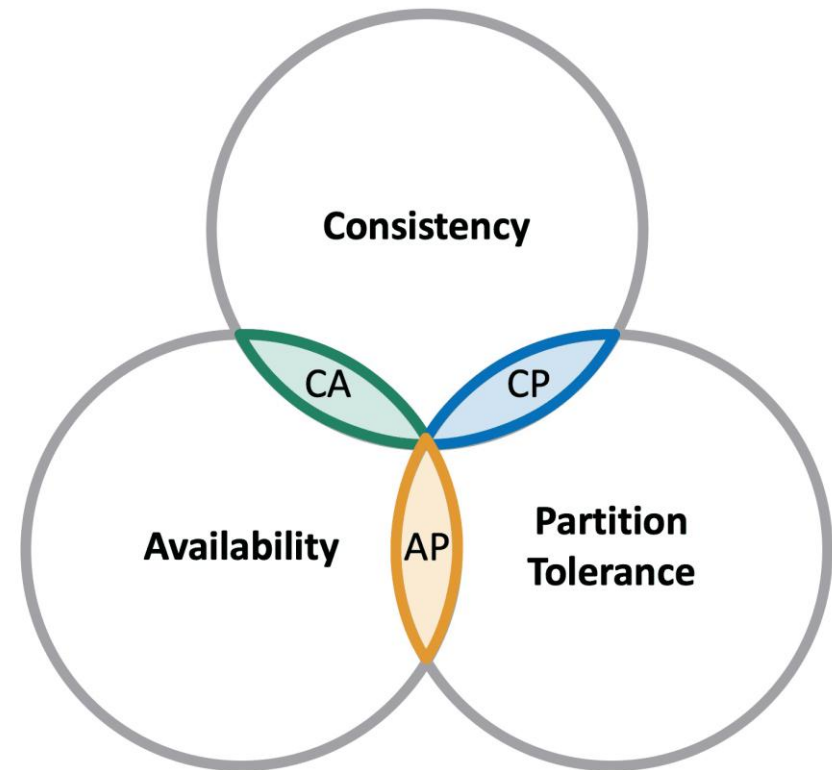
UML Design Class Diagram



CAP Theorem

The two selected properties from the CAP theorem for the system were **Consistency** and **Partition Tolerance**:

- Ensures data consistency even in case of network partition.
- Allows periods of unavailability to occur.



Document DB Design (i)

Collections

User collection

```
{
  "_id" : ObjectId( '6977493f8a21e3569e717b99' ),
  "name" : "Martina",
  "lastname" : "Iacononi",
  "email" : "m.iacononi@studenti.unipi.it",
  "username" : "Marty",
  "password" :
"$2a$10$QTKTtzzL3dgR4SAHXWJR0P0AfBQ0cPIStiBdocPIStiBdoAweZVmg.H8rc3Wm",
  "date_of_birth" : "2002-02-22T23:00:00.000+00:00",
  "interests" : [ "FESTIVAL" ],
  "friends" : [
    "Alex"
  ],
  "home_town": {
    "city_name" : "Cascina",
    "full_address" : "8, Via vecchia fiorentina, Cascina, Pisa, Toscana, 56021",
    "loc" : {
      "type" : "Point",
      "coordinates" : [45.6522964, 8.7945449]
    }
  },
  "reviewd_events": [
    {
      "event_name" : "Porchetta Festival",
      "score" : 5
    }
  ]
}
```

Document DB Design (ii)

Collections

Organizers collection

```
{
  "_id" : ObjectId('6974b78924af76d0657ff218'),
  "list_of_members" : [
    {
      "id" : "6974a15444ac48c0657ac487",
      "name" : "Frank",
      "password" : "$2a$10FiCi1T0teazoz6AZPE76iefh0t9Qgbe864tCrIJztCUNl0zdtgz4e"
    }
  ],
  "pending_request" : [],
  "name" : "Magnifici Tre",
  "VAT_number" : "TR948334764",
  "email" : "magnifici3@gmail.com",
  "events" : [
  ],
  "type" : "ORGANIZATION"
}
```

```
{
  "_id" : ObjectId('6974a15444ac48c0657ac487'),
  "username" : "Frank",
  "date_of_birth" : "2002-02-18T23:00:00.000+00:00",
  "password" : "$2a$10FiCi1T0teazoz6AZPE76iefh0t9Qgbe864tCrIJztCUNl0zdtgz4e",
  "organizations" : [
    {
      "id" : "6974bfd341ac693c0eccc86b",
      "name" : "Magnifici Tre"
    }
  ],
  "name" : "Francesco Della Maggiora",
  "VAT_number" : "FR6Q218643906",
  "email" : "f.dellamaggiora2@studenti.unipi.it",
  "events" : [
    {
      "id" : "697504dd461ac693c0ed6520",
      "name" : "Porchetta Festival"
    }
  ],
  "type" : "ORGANIZER"
}
```


Document DB Design (iii)

Collections

Events collection

```
{
  "_id" : ObjectId('697504dd461ac693c0ed6520'),
  "event_name" : "Porchetta Festival",
  "position" : {
    "city_name" : "Lucca",
    "full_address" : "15, Via Fillungo, Lucca, Toscana, 55100",
    "loc" : {
      "type" : "Point",
      "coordinates" : [45.6522964, 8.7945449]
    }
  },
  "ticket_price" : {
    "Full Price" : 15,
    "Under 12" : 0
  },
  "starting_date" : "2026-03-1T23:00:00.000+00:00",
  "ending_date" : "2026-03-10T23:00:00.000+00:00",
  "categories" : ["FESTIVAL"],
  "description" : "The Porchetta Festival is the number one festival in Italy about ...",
  "url_img" : "https://wips.plus.it/cips/images/local/tnet/eventin/9/4/tnet_7ecf1540",
  "startin_times" : ["18:00"],
  "reviews" : [
    {
      "score" : 5,
      "username" : "Marty",
      "text" : "The best Porchetta that I have ever ate!"
    }
  ],
  "total_review" : 1,
  "event_score" : 5,
  "attendees" : [
    {
      "id" : "6977493f8a21e3569e717b99",
      "email" : "m.icoponi@studenti.unipi.it",
      "username" : "Marty",
      "home_town" : "Cascina",
      "fate_of_birth" : "2002-02-22T23:00:00.000+00:00"
    },
    {
      "id" : "6977484s8a24e7419e567a14",
      "email" : "a0.battisti@studenti.unipi.it",
      "username" : "Alex",
      "home_town" : "Livorno",
      "fate_of_birth" : "2000-09-17T23:00:00.000+00:00"
    }
  ]
}
```

Document DB Design (iv)

Relevant Queries

Statistics Analysis for Events

Calculates events' statistics,
such as:

- **average rating**
- **average attendants' age**
- **predicted income**

```
@Aggregation(pipeline = {
  "{ '$match': {'_id': ?0} }",
  ""
  {
    "$project": {
      "total_attenders": { "$size": { "$ifNull": ["$attendees", []] } },
      "average_rating": { "$ifNull": ["$event_score", 0] },
      "date_update": "$$NOW",
      "publish": { "$literal": false },
      "totalAgeSum": {
        "$reduce": {
          "input": { "$ifNull": ["$attendees", []] },
          "initialValue": 0,
          "in": {
            "$add": [
              "$$value",
              {
                "$dateDiff": {
                  "startDate": { "$toDate": "$$this.date_of_birth" },
                  "endDate": "$$NOW",
                  "unit": "year"
                }
              }
            ]
          }
        }
      }
    }
  }
},
```

Document DB Design (v)

Relevant Queries

Statistics Analysis for Events

```
"calculatedIncome": {
  "$reduce": {
    "input": { "$ifNull": ["$attendees", []] },
    "initialValue": 0,
    "in": {
      "$add": [
        "$$value",
        { "$ifNull": [ { "$getField": { "field": "$$this.ticket_type", "input":
"$ticket_price" } }, 0 ] }
      ]
    }
  },
  "originList": {
    "$map": {
      "input": { "$setUnion": { "$ifNull": ["$attendees.home_town", []] } },
      "as": "city",
      "in": {
        "k": "$$city",
        "v": {
          "$size": {
            "$filter": {
              "input": { "$ifNull": ["$attendees", []] },
              "as": "att",
              "cond": { "$eq": ["$$att.home_town", "$$city"] }
            }
          }
        }
      }
    }
  }
}
"
```

Document DB Design (vi)

Relevant Queries

Statistics Analysis for Events

```
    """
    {
      "$project": {
        "date_update": 1,
        "total_attenders": 1,
        "publish": 1,
        "average_rating": 1,
        "predicted_income": { "$ifNull": ["$calculatedIncome", 0] },
        "average_age": {
          "$cond": [
            { "$eq": ["$total_attenders", 0] },
            0,
            { "$divide": ["$totalAgeSum", { "$cond": [{ "$eq": ["$total_attenders", 0] },
1, "$total_attenders"] }] }
          ]
        },
        "origin_attenders": { "$arrayToObject": "$originList" }
      }
    }
    """
  })
  Statistics calculateStatistics(String eventId);
```

Document DB Design (vii)

Relevant Queries

Organizing Group's Organizers Trend Analysis

Calculates the number of **event each member** of an Organizing Group has created, to compare it to the **total** number of **events** created by the Organizing Group.

```
@Aggregation(pipeline = {
  """
  {
    $match: {
      $or: [
        { "type": "ORGANIZER", "organizations.id": ?0 },
        { "type": "ORGANIZATION", "_id": ?0 }
      ]
    }
  }
  """,
  {
    $group: {
      _id: null,
      organizationTotalEvents: {
        $max: {
          $cond: [
            { $eq: ["$type", "ORGANIZATION"] },
            { $size: { $ifNull: ["$events", []] } },
            0
          ]
        }
      ]
    }
  },
},
```

Document DB Design (viii)

Relevant Queries

Organizations' Organizers Trend Analysis

```
list: {
  $push: {
    $cond: [
      { $eq: ["$type", "ORGANIZER"] },
      {
        username: "$username",
        name: "$name",
        lastname: "$lastname",
        events: "$events"
      },
      "$$REMOVE"
    ]
  }
}
```


Document DB Design (ix)

Relevant Queries

Organizations
' Organizers
Trend
Analysis

```
    ""
    {
      $project: {
        _id: 0,
        analysis: {
          $map: {
            input: "$list",
            as: "user",
            in: {
              username: "$$user.username",
              fullname: { $concat: ["$$user.name", " ", "$$user.lastname"] },
              organizerEventsCount: { $size: { $ifNull: ["$$user.events", []] } },
              organizationTotalEvents: "$organizationTotalEvents"
            }
          }
        }
      }
    }
  }
}
})
List<HashMap<?, ?>> getOrganizationAnalysis(String organizationId);
```

Document DB Design (x)

Relevant Queries

Events' Monthly Demographic

Calculates how many **events** were created for **each month** of a specified year

```
@Aggregation(pipeline = {
  """
  {
    "$match": {
      "position.city_name": ?0,
      "$expr": { "$eq": [ { "$year": "$starting_date" }, ?1 ] }
    }
  }
  """,
  """
  {
    "$group": {
      "_id": { "$month": "$starting_date" },
      "total_count": { "$sum": 1 }
    }
  }
  """,
  """
  {
    "$densify": {
      "field": "_id",
      "range": {
        "step": 1,
        "bounds": [1, 13]
      }
    }
  }
  """,
  """
  {
    "$project": {
      "total_count": 1
    }
  }
  """,
  """
  {
    "$sort": {
      "total_count": -1
    }
  }
  """,
  """
  {
    "$limit": 1
  }
  """,
  """
  {
    "$out": "events_monthly_demographic"
  }
  """
})
```

Document DB Design (xi)

Relevant Queries

Events' Monthly Demographic

```
"{ '$sort': { '_id': 1 } }",
  ""
  {
    "$project": {
      "city": ?0,
      "year": {"$literal": ?1},
      "month": {
        "$arrayElemAt": [
          [ "", "January", "February", "March", "April", "May", "June", "July", "August",
"September", "October", "November", "December" ],
          "$_id"
        ]
      },
      "event_count": { "$ifNull": ["$total_count", 0] },
      "_id": 0
    }
  }
  ""
})
List<HashMap<String, Object>> eventDemographic(String city, int year);
```

MongoDB Replica Set Configuration

- **w = 'majority'**: writes must be acknowledged by the majority of nodes
- **journal = true**: the transactions are appended to the journal on disk
- **readPreference = 'primary'**: reads can only occur on the primary node

IP: 10.1.1.17
mongoDB: PRIMARY

IP: 10.1.1.45
mongoDB: SECONDARY

IP: 10.1.1.44
mongoDB: SECONDARY
Neo4j

MongoDB Replica Set Configuration

Write concern: majority

As the non-functional requirements state, the application *must guarantee data consistency as well as partition tolerance*, leading to this parameter choice *which ensures consistency at the expense of latency*. Also, since the application is read-heavy, *write-derived latency is not a concern*.

Read preference: primary

It's important for users to *read only correct and updated data*, meaning that stale data musn't be shown. For this reason the read preference has to be set to "primary". If the primary node isn't available the operation is rejected, which is consistent with the non-functional requirements.

MongoDB Indexes

Event collection

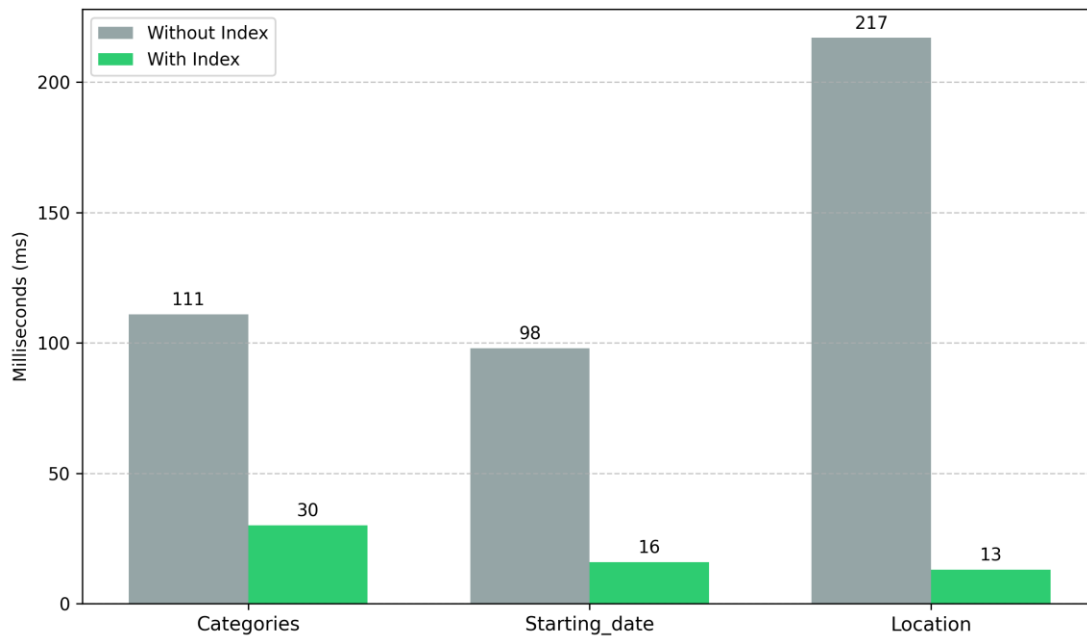
The most frequent read operations is the *searching of events* based on their categories, starting date or location. For this reason the following indexes have been defined:

- **Categories:** the dataset contains few categories and the adding of new category is very rare. This index provides good read performance enhancement.
- **Starting_date:** an even greater improvement is provided by indexing this event field when read operations are performed.
- **Location:** this is the only *geo-spatial index* utilized to query on spatial coordinates. It grants the highest performance increase among every other index.

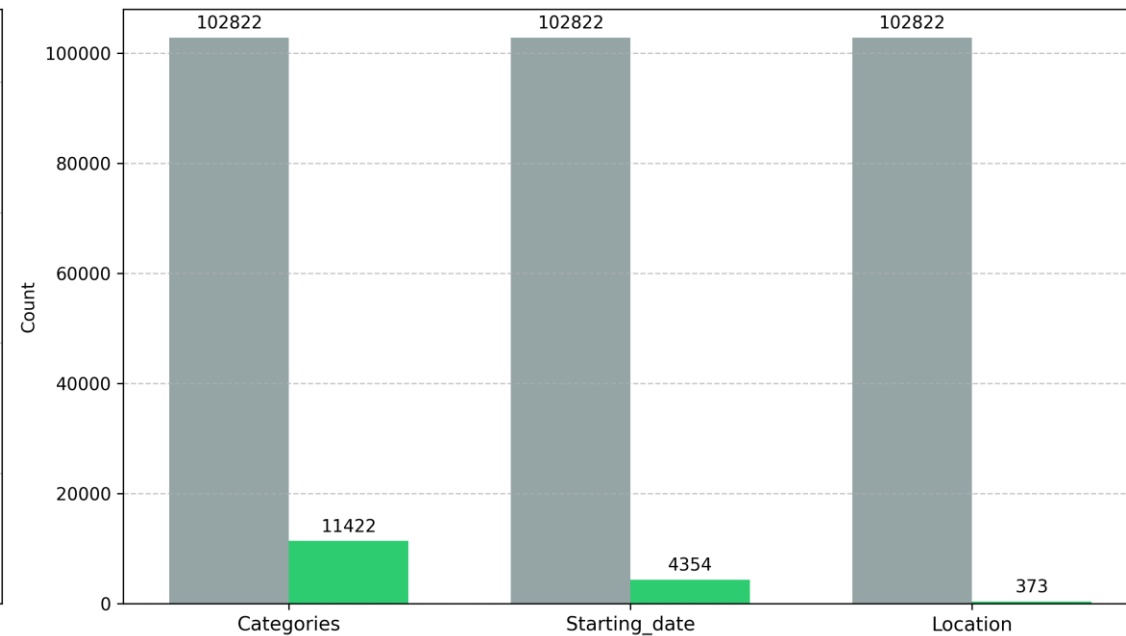
MongoDB Indexes

Database Optimization Results: Impact of Indexing

Execution Time

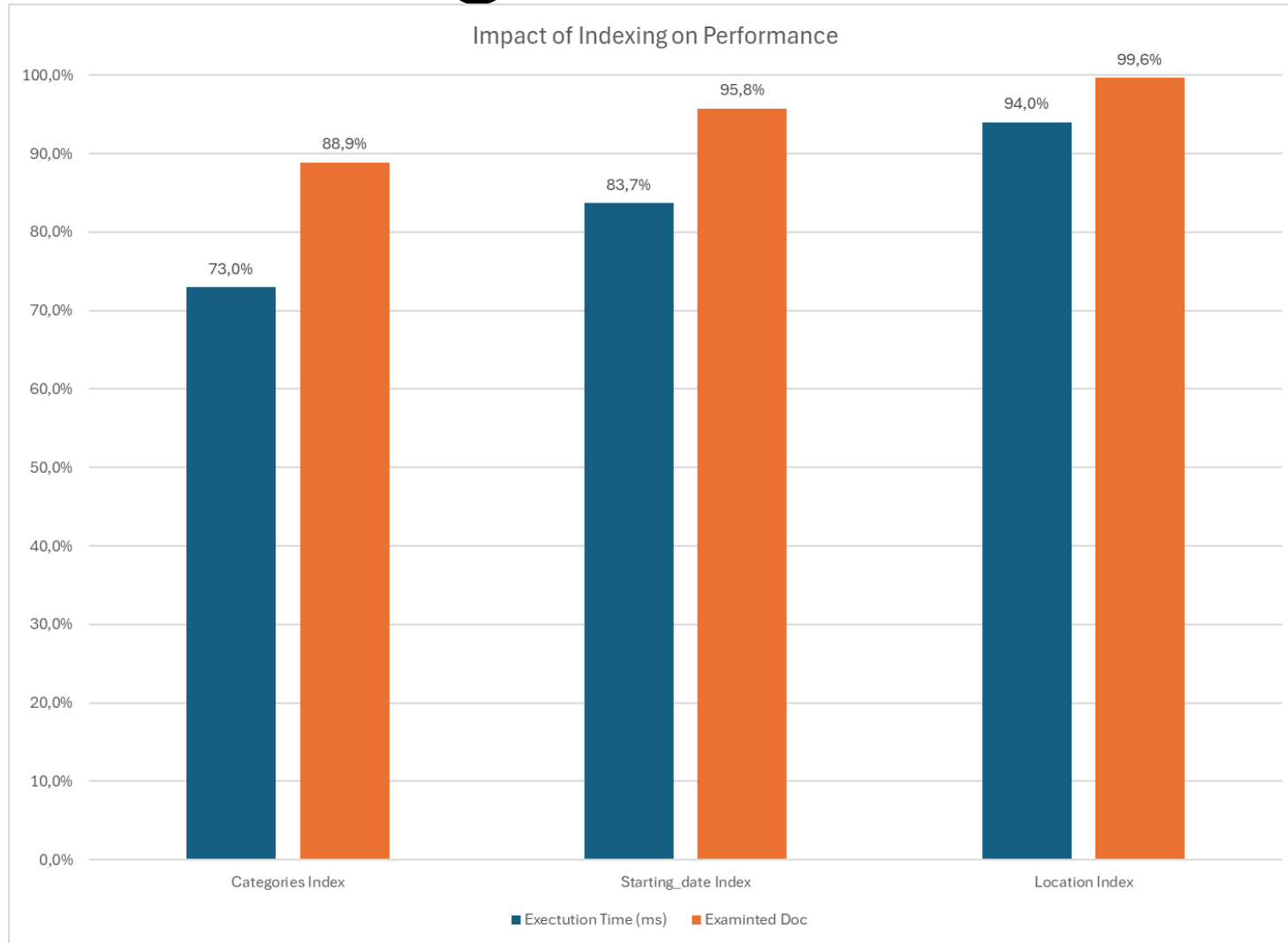


Examined Documents



The lower the better

MongoDB Indexes

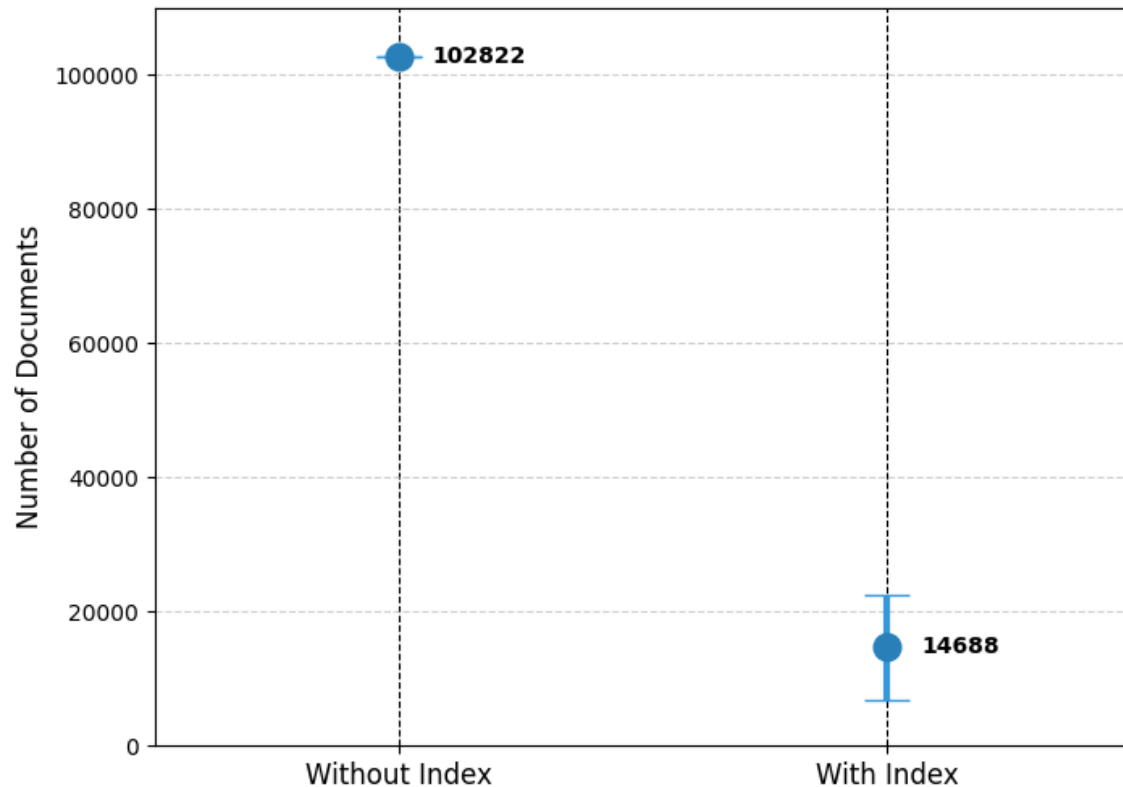


The Higher The Better

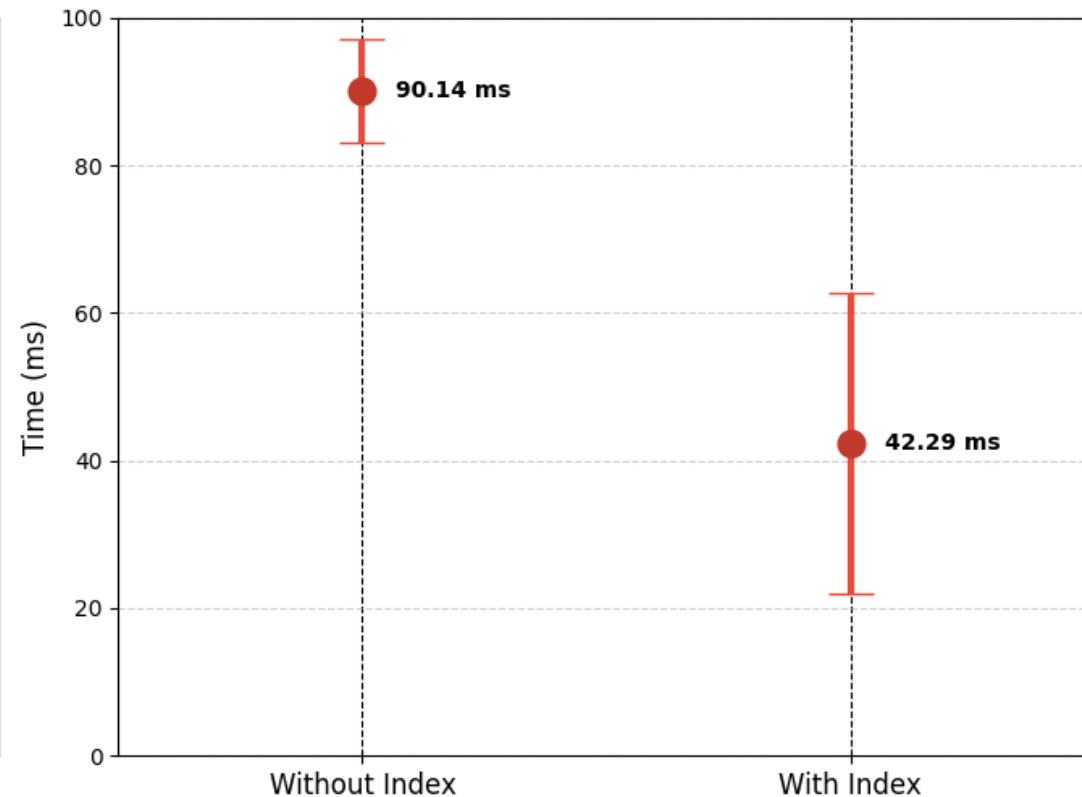
MongoDB Indexes

categories Index Performance

Examined Documents (Confidence Interval 75%)



Execution Time (Confidence Interval 75%)



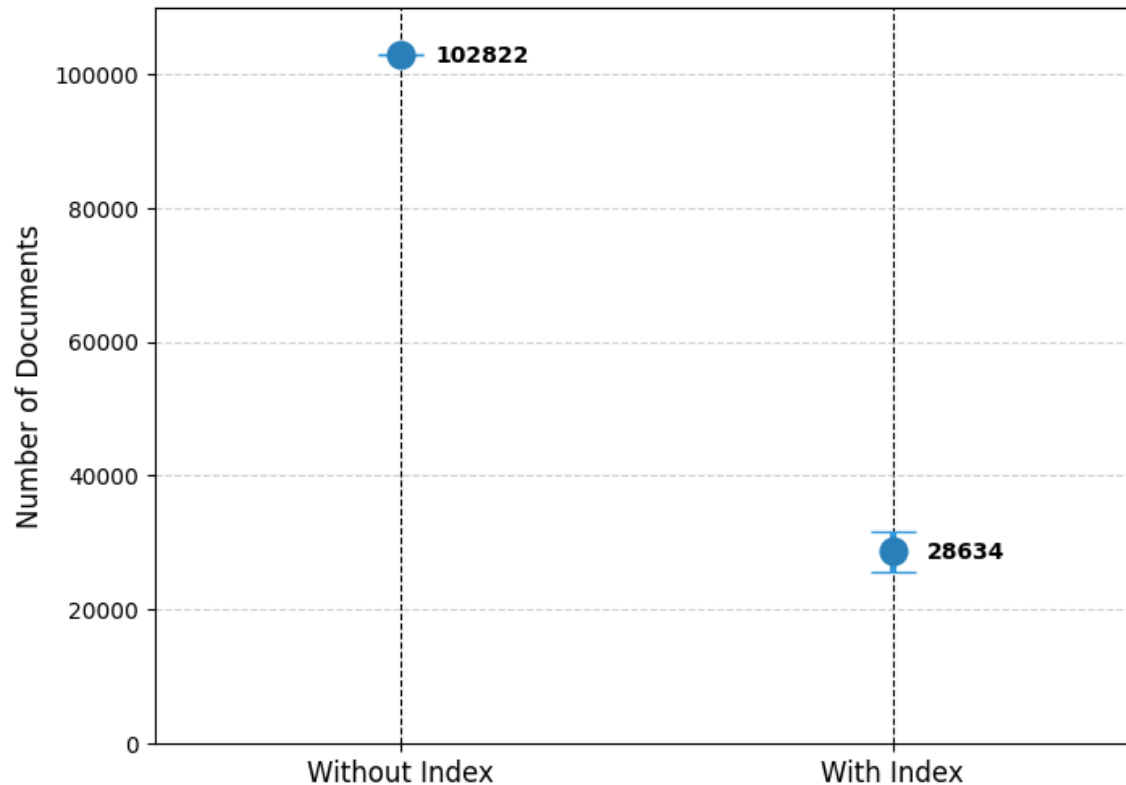
The lower the better

Tested over: ``db.events.find({"categories": {"$in": [$category] }})``

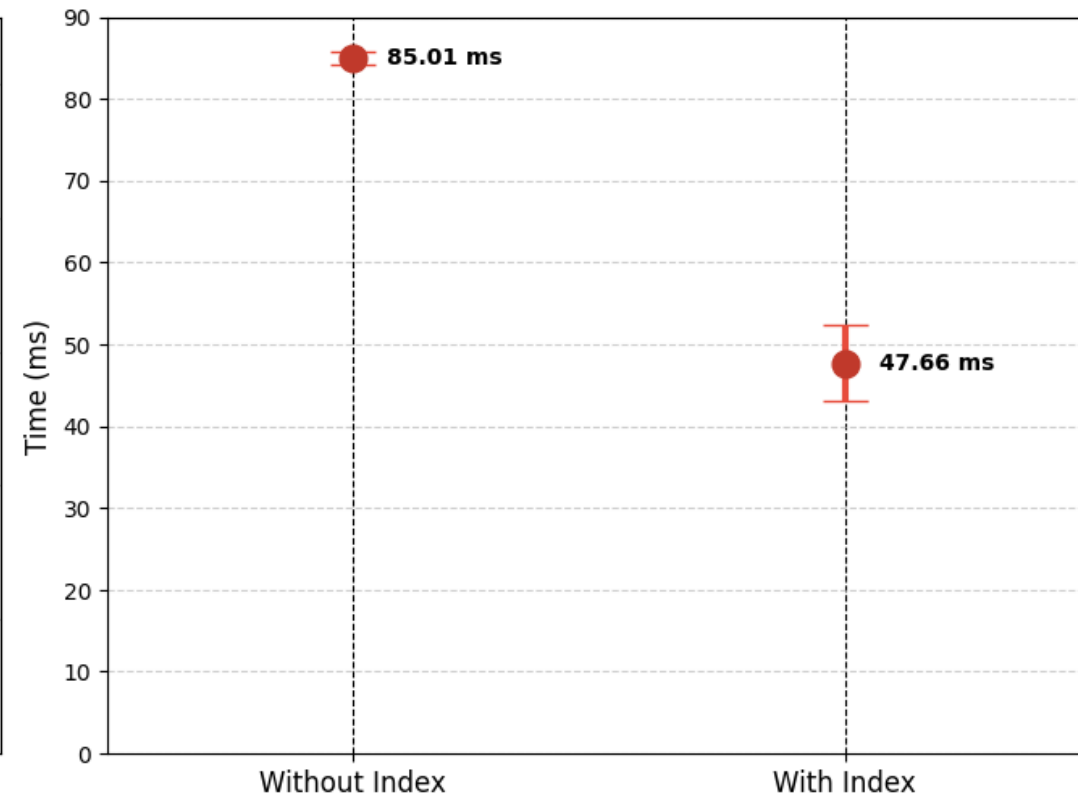
MongoDB Indexes

starting_date Index Performance

Examined Documents (Confidence Interval 75%)



Execution Time (Confidence Interval 75%)



The lower the better

Tested over: ``db.events.find({"starting_date": {"$gte": $date}})``

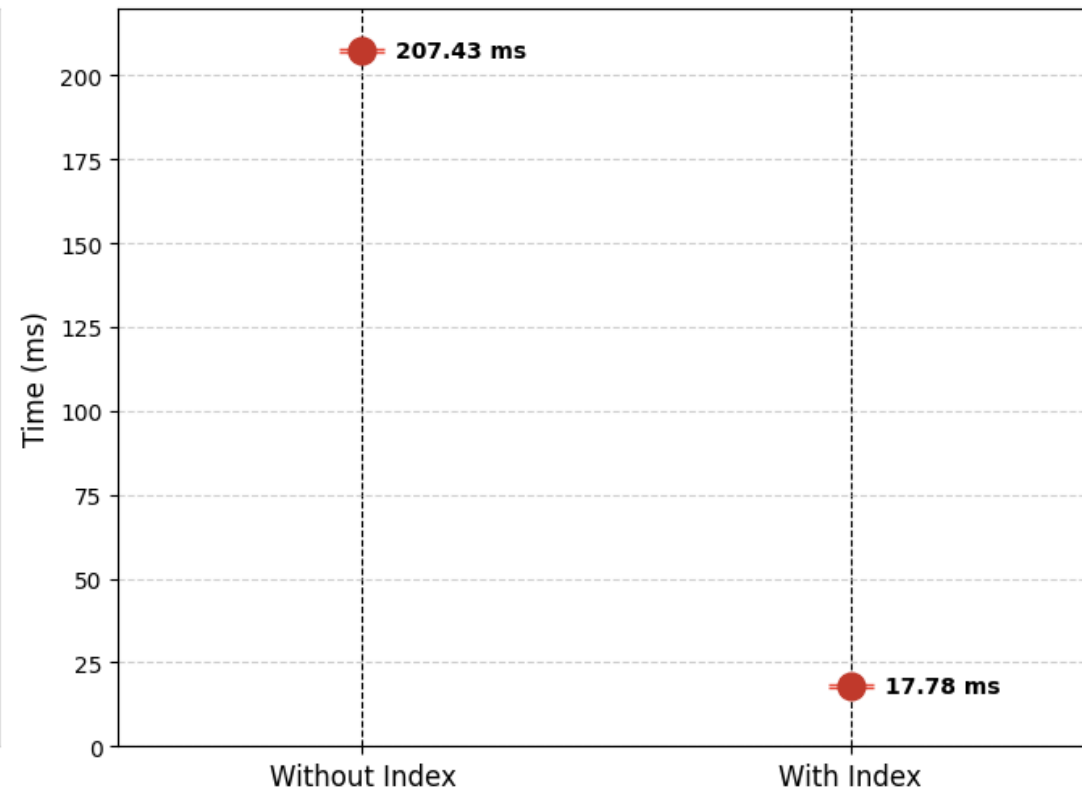
MongoDB Indexes

Geospatial Index Performance

Examined Documents (Confidence Interval 75%)



Execution Time (Confidence Interval 75%)



The lower the better

Without Index Tested over: ``db.events.find({"position.loc": { $geoWithin: { $centerSphere: [$coordinate, 10 / 6378.1 ] } } })``

With Index Tested over: ``db.events.find({"position.loc": { $near: { $geometry: { type: "Point", coordinates: $coordinate }, $maxDistance: 1000 } } })``

Discussion on MongoDB Data Sharding

The best algorithm to use is the **hashing algorithm** and the selected shard key would be the **_id** field for every collection.

This is a ubiquitous field which permits to *shard documents evenly across every shard*, distributing operations load.

Another idea, only regarding events, *was to shard them based on their "macro" location* (northern events, center events and southern events), but such an operation *would require the addition of an ad-hoc field to be performed effectively* and, more importantly, *there's the risk of creating "hot spots"*, meaning that one shard may be related to many more documents than another.

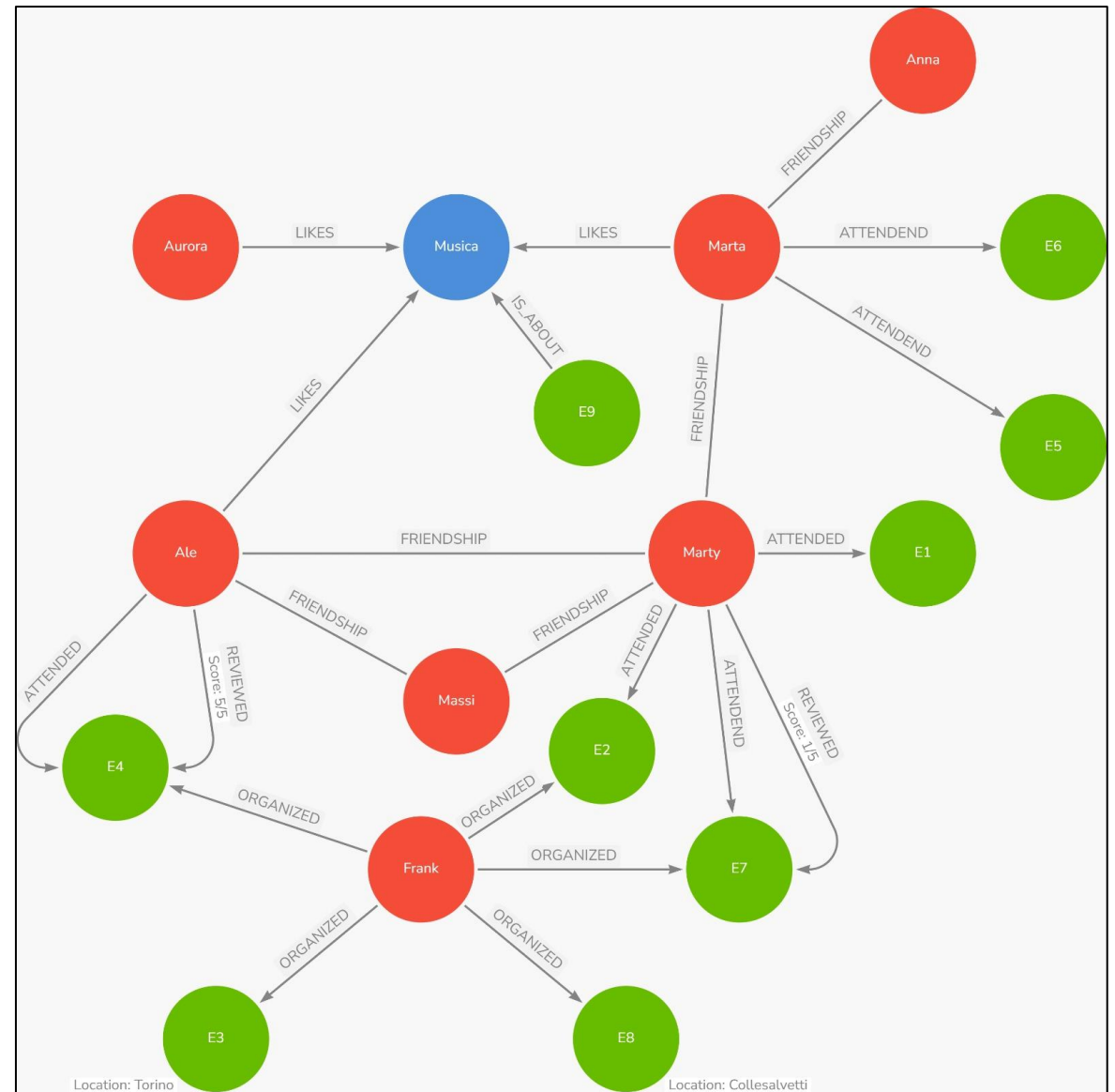
Graph DB Design

Entities:

- User (*id, username, home_coordinates*)
- Event (*id, name, starting_date, ending_date, location_coordinates*)
- Organizer (*id, name*)
- Topic

Relationships:

- LIKES (user – topic)
- IS_ABOUT (event – topic)
- ORGANIZED (organizer – event)
- REVIEWED (user – event)
- ATTENDED (user – event)
- FRIENDSHIP (user – user)



Graph-based Queries (i)

Events Suggestions

Suggests new events to the user, considering:

- events organized by **organizers** whose **events** the user attended already and **scored ≥ 5**
- events based on **topics** the user likes
- events **highly rated** by the user's **friends**

```
@Query("MATCH (user:User {id: $userId})-[review:`:REVIEWS`]->(event:Event)\n" +  
  "WHERE review.score >= 4\n" +  
  "MATCH (event)<-[:`:ORGANIZED`]->(organizer:Organizator)\n" +  
  "MATCH (organizer)-[:`:ORGANIZED`]->(newEvent:Event)\n" +  
  "WHERE NOT (user)-[:`:ATTENDS`|`:REVIEWS`]->(newEvent)\n" +  
  "WITH user, COLLECT(DISTINCT newEvent) as organizerEvents\n" +  
  "MATCH (user)-[:`:LIKES`]->(topic:Topic)\n" +  
  "MATCH (topicEvent:Event)-[:`:IS_ABOUT`]->(topic)\n" +  
  "WHERE NOT (user)-[:`:ATTENDS`|`:REVIEWS`]->(topicEvent)\n" +  
  "WITH user, organizerEvents, COLLECT(DISTINCT topicEvent) AS topicEvents\n" +  
  "WITH user, organizerEvents + topicEvents AS organizerTopicEvents\n" +  
  "MATCH (user)-[:`:FRIENDSHIP`]->(friend:User)\n" +  
  "WITH user, organizerTopicEvents, COLLECT(friend) AS friends\n" +  
  "UNWIND organizerTopicEvents AS event\n" +  
  "OPTIONAL MATCH (friend)-[badFriendReview:`:REVIEWS`]->(event)\n" +  
  "WHERE badFriendReview.score < 2\n" +  
  "WITH user, event, friends, badFriendReview\n" +  
  "WHERE badFriendReview IS NULL\n" +  
  "WITH user, COLLECT(DISTINCT event) as filteredEvents, friends\n" +  
  "MATCH (friend)-[friendReview:`:REVIEWS`]->(friendEvent:Event)\n" +  
  "WHERE friendReview.score >= 4\n")
```

Graph-based Queries (ii)

Events Suggestions

```
"AND NOT (user)-[:`ATTENDS`|`REVIEWS`]->(friendEvent)\n" +  
"WITH user, filteredEvents, COLLECT(DISTINCT friendEvent) AS friendEvents\n" +  
"WITH user, filteredEvents + friendEvents AS allEvents\n" +  
"UNWIND allEvents AS event\n" +  
"WITH user, event, \n" +  
"6371 * 2 * \n" +  
"  atan2(\n" +  
"    sqrt(\n" +  
"      sin(radians(event.coordinates[1] - user.coordinates[1])/2)^2 +\n" +  
"      cos(radians(user.coordinates[1])) * cos(radians(event.coordinates[1])) *\n" +  
"      sin(radians(event.coordinates[0] - user.coordinates[0])/2)^2\n" +  
"    ),\n" +  
"    sqrt(\n" +  
"      1 - (\n" +  
"        sin(radians(event.coordinates[1] - user.coordinates[1])/2)^2 +\n" +  
"        cos(radians(user.coordinates[1])) * cos(radians(event.coordinates[1])) *\n" +  
"        sin(radians(event.coordinates[0] - user.coordinates[0])/2)^2\n" +  
"      )\n" +  
"    )\n" +  
"  ) AS distanceKm\n" +  
"WHERE distanceKm <= $distanceKm\n" +  
"AND event.startingDate > $startingDate\n" +  
"WITH event.eventName AS name, head(collect(event)) AS uniqueEvent\n" +  
"RETURN uniqueEvent\n" +  
"ORDER BY uniqueEvent.startingDate\n" +  
"LIMIT $limit")  
List<EventNode> suggestEvent(String userId, int distanceKm, LocalDate startingDate, int limit);
```

Graph-based Queries (iii)

Friends Suggestions

Suggests new friends,
based on:

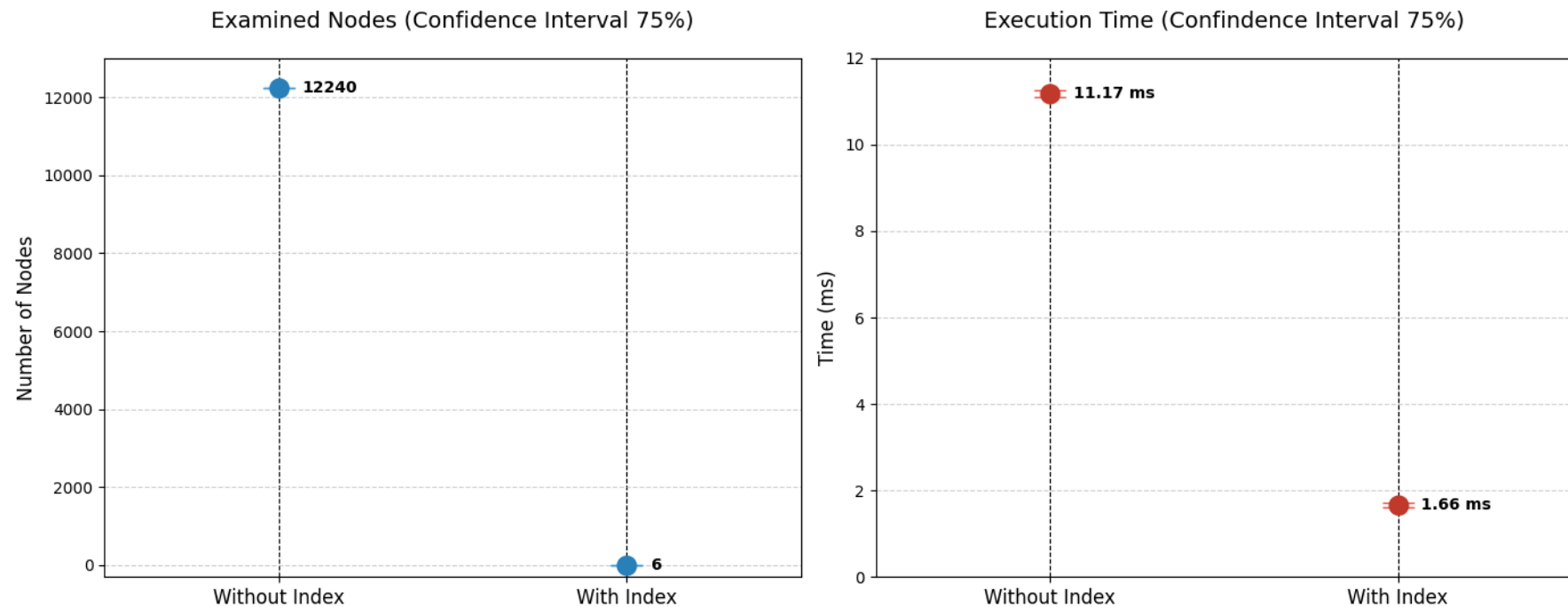
- **friends** of the user's **friends**
- users with **shared** topics of interest

```
@Query("MATCH (user:User {id: $userId})-[:`FRIENDSHIP`]- (friend:User)\n" +  
      "MATCH (friend)-[:`FRIENDSHIP`]- (newFriend:User)\n" +  
      "WHERE NOT (user)-[:`FRIENDSHIP`]- (newFriend)\n" +  
      "WITH user, COLLECT (DISTINCT newFriend) AS newFriends\n" +  
      "MATCH (user)-[:`LIKES`]->(topic:Topic)\n" +  
      "MATCH (topic)<-[:`LIKES`]- (topicFriend:User)\n" +  
      "WHERE user.id <> topicFriend.id\n" +  
      "AND NOT (user)-[:`FRIENDSHIP`]- (topicFriend)\n" +  
      "WITH newFriends, COLLECT (DISTINCT topicFriend) AS topicFriends\n" +  
      "WITH newFriends + topicFriends AS allFriends\n" +  
      "UNWIND allFriends as friend\n" +  
      "RETURN DISTINCT friend\n" +  
      "LIMIT $limit;")  
List<UserNode> suggestFriends(String userId, int limit);
```

Neo4j Index

An index on user nodes' id has been defined as the application's graph queries always begin from a user node.

User Index Performance

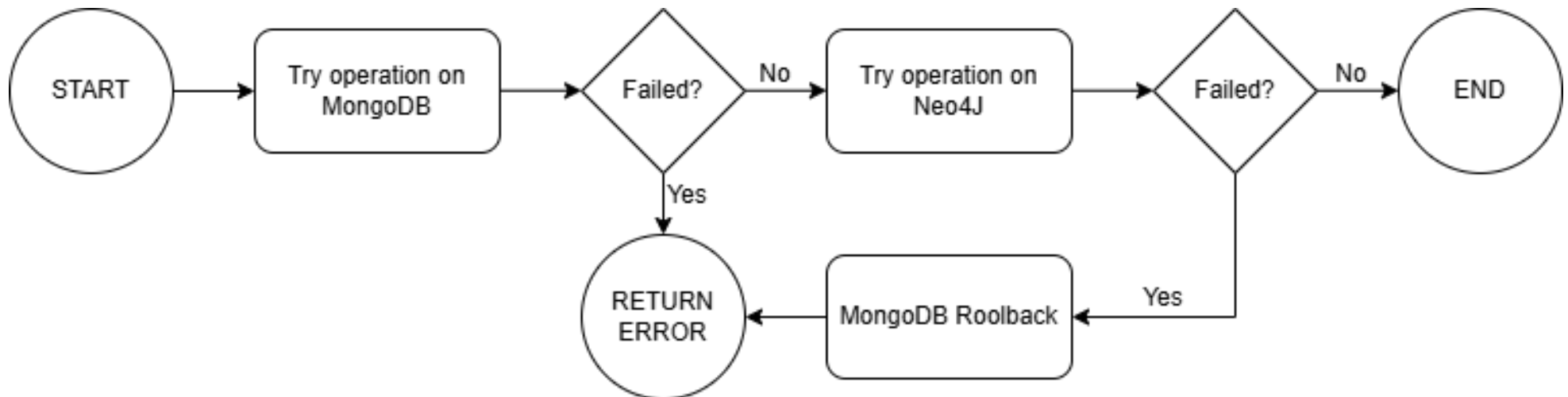


The lower the better

Tested over: `Match (u:User {id: \$id}) return u`

Handling Intra-DB Consistency

Hypothesis: since MongoDB have 3 replicas, we can assume that a fail on connection is almost impossible.



Swagger UI REST APIs documentation

Swagger doc url: <http://10.1.1.17:8080/swagger-ui/index.html>

Organizer Operations Endpoints for actions performed by authenticated Organizers

PUT `/organizer/update` Updated an Organizer data

POST `/organizer/registerOrganization` Register a new managed Organization

POST `/organizer/registerEvent` Create a new Event

GET `/organizer` Retrieve all organizers

GET `/organizer/me` Get organizer by ID

DELETE `/organizer/organization/{organizationName}` Removes a membership from an Organization

Javadoc code documentation

Javadoc url: <http://10.1.1.17:8080/doc/index.html>

OVERVIEW	PACKAGE	CLASS	USE	TREE	INDEX	HELP
----------	---------	-------	-----	------	-------	------

TonightsCall 0.0.1-SNAPSHOT API

Packages

Package	Description
it.unipi.tonightscall	
it.unipi.tonightscall.controller	
it.unipi.tonightscall.DTO	
it.unipi.tonightscall.entity.document	
it.unipi.tonightscall.entity.graph	
it.unipi.tonightscall.jwt	
it.unipi.tonightscall.repository.document	
it.unipi.tonightscall.repository.graph	
it.unipi.tonightscall.security	
it.unipi.tonightscall.service	
it.unipi.tonightscall.utilies	

Live Demo with Postman

